

Corporate Multi-Vendor Order Tracking Platform

This document contains complete enterprise-level architecture, AWS deployment planning, microservices design, Android structure, JWT authentication flow, development timeline, team estimation, costing, sprint planning, and production-grade recommendations.

1. Project Overview

- Multi-client and multi-vendor service marketplace platform
- Android applications for customers and vendors
- Admin dashboard for centralized management
- Real-time order tracking similar to delivery platforms
- WhatsApp order sharing to vendors
- Subscription-based business model
- Payment gateway integration

2. Recommended Technology Stack

- Android: Kotlin + Jetpack Compose
- Backend: Java 17 + Spring Boot 3
- Database: PostgreSQL
- Authentication: JWT + Spring Security
- Cloud Platform: AWS
- Storage: AWS S3
- Push Notifications: Firebase
- CI/CD: GitHub Actions
- Containerization: Docker

3. Microservice Architecture

- API Gateway Service
- Authentication Service
- Vendor Service
- Order Service
- Payment Service
- Notification Service
- Subscription Service
- Tracking Service
- Admin Service

4. AWS Deployment Architecture

- Application Load Balancer
- EC2 or ECS deployment
- RDS PostgreSQL database
- S3 for file storage
- CloudWatch monitoring
- Route53 for DNS
- SSL certificates using ACM
- Auto Scaling Groups

5. JWT Authentication Flow

- User login with email/mobile
- Credential validation
- JWT token generation
- Refresh token mechanism
- Secure API access using Spring Security
- Role-based authorization

6. Android Application Structure

- presentation/
- domain/
- data/
- repository/
- network/
- firebase/
- di/
- utils/

7. Backend Folder Structure

- controller/
- service/
- repository/
- entity/
- dto/
- security/

- config/
- exception/
- util/

8. Main Database Tables

- users
- roles
- vendors
- services
- orders
- order_tracking
- payments
- subscriptions
- notifications
- reviews

9. API Design Structure

- POST /auth/login
- POST /auth/register
- GET /vendors
- POST /orders
- GET /orders/{id}
- POST /payments/initiate
- GET /tracking/{orderId}

10. Production-Grade Features

- Audit logging
- Centralized exception handling
- API versioning
- Rate limiting
- Retry mechanism
- Redis caching
- Monitoring and alerting
- Scalable architecture

11. Team Requirement Estimation

- Android Developer: 1-2

- Backend Developer: 1-2
- Frontend/Admin Developer: 1
- QA Engineer: 1
- DevOps Engineer: Part-time
- UI/UX Designer: 1

12. Estimated Development Timeline

- Requirement Gathering: 5 Days
- UI/UX Design: 7-10 Days
- Backend Development: 20-30 Days
- Android Development: 25-35 Days
- Admin Panel Development: 10-15 Days
- Payment Gateway Integration: 5 Days
- WhatsApp Integration: 3-5 Days
- AWS Deployment: 3-5 Days
- Testing & QA: 10-15 Days
- Production Release: 2-3 Days

13. Total Project Completion Estimation

- 2 Developers Team: 90-120 Days
- 3 Developers Team: 60-75 Days
- 5+ Developers Team: 45-60 Days
- Enterprise Product Timeline: 4-6 Months

14. Sprint Planning (60 Days)

- Sprint 1: Requirement analysis and architecture
- Sprint 2: Authentication and user management
- Sprint 3: Vendor and service modules
- Sprint 4: Order management and tracking
- Sprint 5: Payment gateway and subscriptions
- Sprint 6: Notifications and WhatsApp integration
- Sprint 7: Testing, optimization, deployment

15. Estimated Monthly Infrastructure Cost

- AWS Infrastructure: ■10,000 - ■15,000/month
- Payment Gateway Charges: Transaction based
- WhatsApp API Charges: Usage based

- Firebase: Mostly free initially
- Domain & SSL: Annual cost

16. Recommended DevOps Pipeline

- GitHub Repository
- GitHub Actions CI/CD
- Docker image build
- Push to AWS ECR
- Deploy to ECS/EC2
- CloudWatch monitoring

17. High Level Architecture

Android App → API Gateway → Microservices → PostgreSQL / Redis / AWS S3

18. Low Level Design Recommendations

Use layered architecture with DTO mapping, repository pattern, centralized exception handling, JWT filters, Swagger documentation, and environment-based configuration.